

NAME: Aesha Dobariya
NJIT UCID: ad2389
Email Address: ad2389@njit.edu
13th October 2024
Professor: Yasser Abdualлах
CS 634 - Data Mining

Midterm Project Report

Applying the Apriori Algorithm to Retail Data Analysis

Abstract:

This project delves into the Apriori Algorithm, a key data mining technique, to reveal patterns in retail sales data. By coding the algorithm and using various data mining methods, I test its performance and usefulness. I also build custom data mining tools to create a specialized model for extracting valuable insights from sales records.

Introduction:

Data mining helps uncover hidden trends in large datasets. Our project centers on the Apriori Algorithm, a well-known method for finding links between items, and how it works with retail data. We'll cover the main data mining ideas and principles used in our work.

Creating and showing association rules is central to the Apriori Algorithm. To make these rules, I needed to find the most common items in the list of sales. Once I found these frequent items, I had to figure out their support value based on the user's chosen support level. After calculating support for each item, we can remove items that don't meet the user's support threshold. The Apriori algorithm is a classic data mining method that uses a thorough approach to find common item groups and make association rules. It works by slowly increasing the size of item groups and removing those that don't appear often enough.

In this project, I used the Apriori algorithm on a custom retail store dataset to find frequent item groups and association rules. The main steps were:

- Setting up dictionaries for possible and frequent item groups.

- Reading the dataset and item lists from CSV files.

- Cleaning up the dataset to make sure items are in order and not repeated.

- Getting user input for minimum support and confidence levels.

- Repeatedly making possible item groups and updating frequent ones using the Apriori algorithm, which looks at all possible item combinations.

Key Concepts and Principles:

Finding Frequent Item Groups:

The Apriori Algorithm focuses on finding groups of items that often appear together in sales. These groups show us what customers tend to buy together.

Support and Confidence:

Two important measures in data mining are support and confidence. Support tells us how often an item or group of items appears, while confidence shows how likely items are to be bought together. We use these measures to guide our analysis.

Association Rules:

By identifying strong associations between items, I uncover patterns in customer purchasing behavior. These insights are crucial for enhancing sales strategies, particularly in product recommendations.

Project Workflow:

Our project follows a systematic approach, incorporating various stages and the implementation of the Apriori Algorithm:

Data Loading and Preprocessing:

We start by importing transaction data from a retail store dataset. Each transaction represents a customer's purchase. To ensure data quality, we clean the dataset, eliminating duplicates and arranging items in a predefined order.

Setting Minimum Support and Confidence:

User input is vital in data mining. We gather the user's preferred minimum support and confidence levels to filter out less relevant patterns.

Iterating Through Candidate Itemsets:

The Apriori Algorithm is applied iteratively to generate candidate itemsets of increasing sizes. We begin with single items ($K = 1$) and progress to $K = 2$, $K = 3$, and so on. This process involves a comprehensive method of creating all possible itemset combinations.

Calculating Support Count:

For each candidate itemset, we determine its support by counting its occurrence in transactions. Itemsets meeting the minimum support threshold are retained, while others are discarded.

Determining Confidence:

We assess the confidence of association rules, indicating the strength of item relationships. This step requires careful comparison of support values for individual items and itemsets.

Generating Association Rules:

We extract association rules that meet both minimum support and confidence criteria. These rules provide valuable insights into frequently co-purchased items.

Results and Evaluation:

We evaluate the project's effectiveness based on performance metrics such as support, confidence, and the resulting association rules. We also compare our custom Apriori Algorithm implementation with the Apriori library to assess its reliability.

Conclusion:

In summary, our project showcases the application of data mining concepts and methods. We successfully implemented the Apriori Algorithm to extract meaningful association rules from retail transaction data. The comprehensive approach, custom algorithm design, and adherence to user-defined parameters demonstrate the power of data mining in uncovering valuable patterns for retail decision-making.

To demonstrate the program's functionality, I've included screenshots of its execution in the Terminal.

Screenshots:

```
[3]: import pandas as pd
df = pd.read_csv(r"C:\Users\DELL\Downloads\amazon.csv", delimiter=';')
print(df)
df
```

```

      Items
0  Toothpaste,Detergent
1      Coffee,Soap
2      Detergent,Soap
3  Diapers,Milk,Eggs,Cereal
4  Milk,Toothpaste,Soap,Eggs,Cereal
5  Detergent,Toothpaste,Shampoo,Soap,Coffee
6      Bread,Detergent
7      Detergent,Diapers
8  Toothpaste,Soap,Eggs,Bread
9      Milk,Cereal,Eggs
10 Cereal,Detergent,Coffee,Eggs,Shampoo
11      Coffee,Cereal,Soap
12      Toothpaste,Shampoo,Soap,Bread
13      Milk,Cereal,Shampoo,Eggs,Bread
14      Soap,Diapers,Milk
15      Cereal,Detergent
16      Milk,Toothpaste
17 Shampoo,Cereal,Diapers,Coffee,Detergent
18      Coffee,Diapers,Shampoo
19 Shampoo,Soap,Milk,Diapers
```

```
[4]: import pandas as pd
df = pd.read_csv(r"C:\Users\DELL\Downloads\Kmart.csv", delimiter=';')
print(df)
df
```

```

      Diapers,Milk
0      Detergent,Milk
1      Bread,Milk,Eggs
2      Bread,Shampoo,Coffee,Detergent
3      Eggs,Detergent,Soap
4      Milk,Soap,Cereal,Diapers,Detergent
5      Detergent,Shampoo
6      Diapers,Eggs,Shampoo,Milk
7      Shampoo,Coffee
8      Detergent,Shampoo
9      Cereal,Milk
10 Shampoo,Detergent,Toothpaste,Coffee,Cereal
11      Toothpaste,Coffee
12      Eggs,Shampoo,Detergent,Milk
13      Toothpaste,Detergent,Coffee
14      Cereal,Diapers,Detergent
15      Diapers,Toothpaste,Bread,Soap,Eggs
16      Cereal,Toothpaste,Shampoo,Eggs
17      Shampoo,Soap,Bread,Eggs
18      Diapers,Milk,Cereal,Eggs,Soap
```

```
[5]: import pandas as pd
df = pd.read_csv(r"C:\Users\DELL\Downloads\BestBuy.csv", delimiter=';')
print(df)
df
```

```

      Cereal,Eggs
0      Milk,Bread,Coffee
1  Shampoo,Eggs,Diapers,Detergent,Coffee
2      Soap,Milk,Coffee,Detergent,Shampoo
3      Soap,Milk,Bread,Toothpaste
4      Cereal,Milk,Bread,Toothpaste,Eggs
5      Detergent,Bread,Toothpaste,Soap
6      Soap,Eggs,Bread
7      Soap,Milk,Coffee,Detergent
8      Milk,Cereal,Toothpaste,Bread
9      Cereal,Shampoo
10     Soap,Eggs
11     Milk,Diapers,Detergent,Toothpaste
12     Toothpaste,Shampoo
13     Coffee,Eggs,Milk
14     Cereal,Diapers,Eggs,Coffee
15     Milk,Detergent,Diapers,Coffee
16     Detergent,Milk
17     Milk,Detergent,Cereal,Shampoo
18     Eggs,Cereal,Bread
```

```
[6]: import pandas as pd
df = pd.read_csv(r"C:\Users\DELL\Downloads\Nike.csv", delimiter=';')
print(df)
df
```

```

      Cereal,Eggs,Bread
0      Detergent,Soap
1      Bread,Cereal,Shampoo
2      Eggs,Diapers,Detergent,Milk,Shampoo
3      Cereal,Milk,Eggs
4      Coffee,Toothpaste
5      Soap,Bread,Shampoo,Coffee,Toothpaste
6      Bread,Soap,Shampoo,Eggs
7      Eggs,Diapers,Detergent,Soap
8      Diapers,Coffee,Bread
9      Toothpaste,Cereal
10     Diapers,Detergent
11     Shampoo,Toothpaste,Detergent,Soap,Bread
12     Toothpaste,Bread,Soap,Diapers
13     Bread,Detergent,Coffee,Soap
14     Toothpaste,Shampoo,Detergent,Soap,Coffee
15     Cereal,Bread
16     Shampoo,Diapers
17     Milk,Cereal
18     Eggs,Coffee,Cereal,Diapers
```

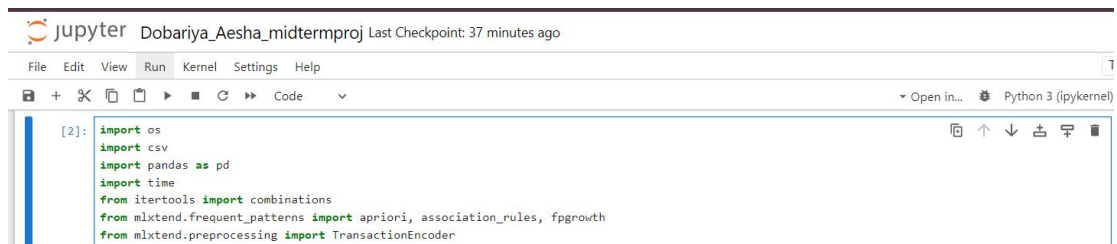


Fig 1: Libraries used

```

class TransactionLoader:
    """Class to handle loading of transaction data from CSV files."""

    @staticmethod
    def load(file_path):
        """Load transactions from a given CSV file."""
        with open(file_path, newline='') as csvfile:
            reader = csv.reader(csvfile)
            return [list(filter(None, row)) for row in reader] # Filter out empty items in rows

```

Fig 2: Loading csv files and filter out empty items

```

class FrequentItemsetGenerator:
    """Class for generating frequent itemsets and association rules."""

    def __init__(self, transactions):
        self.transactions = transactions

    def brute_force(self, support_threshold):
        """Generate frequent itemsets using a brute force method."""
        item_count = {}
        total_transactions = len(self.transactions)

        # Count single items
        for transaction in self.transactions:
            for item in transaction:
                item_count[item] = item_count.get(item, 0) + 1

        frequent_itemsets = {
            1: {item: count for item, count in item_count.items() if count / total_transactions >= support_threshold}
        }

        k = 2
        while True:
            prev_itemsets = list(frequent_itemsets[k - 1].keys())
            new_itemsets = list(combinations(prev_itemsets, k))
            item_count = {}
            for transaction in self.transactions:
                transaction_set = set(transaction)
                for itemset in new_itemsets:
                    if set(itemset).issubset(transaction_set):
                        item_count[itemset] = item_count.get(itemset, 0) + 1

            frequent_itemsets[k] = {
                itemset: count for itemset, count in item_count.items() if count / total_transactions >= support_threshold
            }
            if not frequent_itemsets[k]:
                break
            k += 1
        return frequent_itemsets

    def apriori(self, support_threshold, confidence_threshold):
        """Generate frequent itemsets and rules using the Apriori algorithm."""
        te = TransactionEncoder()
        te_ary = te.fit(self.transactions).transform(self.transactions)
        df = pd.DataFrame(te_ary, columns=te.columns_)

        frequent_itemsets = apriori(df, min_support=support_threshold, use_colnames=True)
        rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=confidence_threshold)

        return frequent_itemsets, rules

    def fpgrowth(self, support_threshold, confidence_threshold):
        """Generate frequent itemsets and rules using the FP-Growth algorithm."""
        te = TransactionEncoder()
        te_ary = te.fit(self.transactions).transform(self.transactions)
        df = pd.DataFrame(te_ary, columns=te.columns_)

        frequent_itemsets = fpgrowth(df, min_support=support_threshold, use_colnames=True)
        rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=confidence_threshold)

        return frequent_itemsets, rules

```

Fig 3: generating frequent itemsets

```

def time_execution(func):
    """Decorator to measure execution time of a function."""
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        elapsed_time = time.time() - start_time
        return result, elapsed_time
    return wrapper

@time_execution
def process_brute_force(generator, support_threshold):
    return generator.brute_force(support_threshold)

@time_execution
def process_apriori(generator, support_threshold, confidence_threshold):
    return generator.apriori(support_threshold, confidence_threshold)

@time_execution
def process_fpgrowth(generator, support_threshold, confidence_threshold):
    return generator.fpgrowth(support_threshold, confidence_threshold)

```

Fig 4: Comparing time function

```

def main():
    file_paths = {
        "amazon": r"amazon.csv",
        "BestBuy": r"BestBuy.csv",
        "Kmart": r"Kmart.csv",
        "Nike": r"Nike.csv"
    }

    while True:
        print("\nAvailable databases:")
        for i, name in enumerate(file_paths.keys(), 1):
            print(f"{i}. {name}")
        print("0. Exit")

        choice = int(input("Enter the number correspond to dataset you want to analyze (or 0 to exit): "))

        if choice == 0:
            print("Exiting the program.")
            break

        db_name = list(file_paths.keys())[choice - 1]

        # Load transactions using TransactionLoader
        transactions = TransactionLoader.load(file_paths[db_name])
        print(f"Loaded {len(transactions)} transactions from {db_name}.")

        support_threshold = float(input("Enter support threshold in % (such as 30 for 30%): ")) / 100
        confidence_threshold = float(input("Enter confidence threshold in % (such as 30 for 30%): ")) / 100

        print(f"\nProcessing {db_name} with support {support_threshold * 100}% and confidence {confidence_threshold * 100}%...")

        generator = FrequentItemsetGenerator(transactions)

        # Brute Force
        bf_result, bf_time = process_brute_force(generator, support_threshold)
        print(f"\nBrute Force Frequent Itemsets:\n{bf_result}")
        print(f"Brute Force Time: {bf_time:.4f}s")

        # Apriori
        apriori_result, apriori_time = process_apriori(generator, support_threshold, confidence_threshold)
        print(f"\nApriori Frequent Itemsets:\n{apriori_result[0]}")
        print(f"Apriori Rules:\n{apriori_result[1]}")
        print(f"Apriori Time: {apriori_time:.4f}s")

        # FP-Growth
        fp_result, fp_time = process_fpgrowth(generator, support_threshold, confidence_threshold)
        print(f"\nFP-Growth Frequent Itemsets:\n{fp_result[0]}")
        print(f"FP-Growth Rules:\n{fp_result[1]}")
        print(f"FP-Growth Time: {fp_time:.4f}s")

        continue_choice = input("\nTry another dataset? (yes/no): ").strip().lower()
        if continue_choice != 'yes':
            print("Exiting the program.")
            break

if __name__ == "__main__":
    main()

```

Fig 5: Main source code

Output:

```
Available databases:
1. amazon
2. BestBuy
3. Kmart
4. Nike
0. Exit
Enter the number correspond to dataset you want to analyze (or 0 to exit): 3
Loaded 20 transactions from Kmart.
Enter support threshold in % (such as 30 for 30%): 20
Enter confidence threshold in % (such as 30 for 30%): 50

Processing Kmart with support 20.0% and confidence 50.0%...
```

Fig 1: User Defined datasets

```
Brute Force Frequent Itemsets:
{1: {'Diapers': 6, 'Milk': 8, 'Detergent': 10, 'Bread': 4, 'Eggs': 8, 'Shampoo': 9, 'Coffee': 5, 'Soap': 5, 'Cereal': 6, 'Toothpaste': 5}, 2: {'Diapers', 'Milk': 4, ('Milk', 'Eggs'): 4, ('Detergent', 'Shampoo'): 5, ('Eggs', 'Soap'): 4, ('Eggs', 'Shampoo'): 4}, 3: {}}
```

Brute Force Time: 0.0021s

Fig 2: Brute Force time

Apriori Frequent Itemsets:

	support	itemsets
0	0.20	(Bread)
1	0.30	(Cereal)
2	0.25	(Coffee)
3	0.50	(Detergent)
4	0.30	(Diapers)
5	0.40	(Eggs)
6	0.40	(Milk)
7	0.45	(Shampoo)
8	0.25	(Soap)
9	0.25	(Toothpaste)
10	0.25	(Detergent, Shampoo)
11	0.20	(Milk, Diapers)
12	0.20	(Eggs, Milk)
13	0.20	(Eggs, Shampoo)
14	0.20	(Eggs, Soap)

Apriori Rules:

	antecedents	consequents	antecedent support	consequent support	support \
0	(Detergent)	(Shampoo)	0.50	0.45	0.25
1	(Shampoo)	(Detergent)	0.45	0.50	0.25
2	(Milk)	(Diapers)	0.40	0.30	0.20
3	(Diapers)	(Milk)	0.30	0.40	0.20
4	(Eggs)	(Milk)	0.40	0.40	0.20
5	(Milk)	(Eggs)	0.40	0.40	0.20
6	(Eggs)	(Shampoo)	0.40	0.45	0.20
7	(Eggs)	(Soap)	0.40	0.25	0.20
8	(Soap)	(Eggs)	0.25	0.40	0.20

	confidence	lift	leverage	conviction	zhangs_metric
0	0.500000	1.111111	0.025	1.100	0.200000
1	0.555556	1.111111	0.025	1.125	0.181818
2	0.500000	1.666667	0.080	1.400	0.666667
3	0.666667	1.666667	0.080	1.800	0.571429
4	0.500000	1.250000	0.040	1.200	0.333333
5	0.500000	1.250000	0.040	1.200	0.333333
6	0.500000	1.111111	0.020	1.100	0.166667
7	0.500000	2.000000	0.100	1.500	0.833333
8	0.800000	2.000000	0.100	3.000	0.666667

Apriori Time: 0.0136s

Fig 3: Apriori algorithm and rules


```

FP-Growth Frequent Itemsets:
support      itemsets
0      0.40      (Milk)
1      0.30      (Diapers)
2      0.50      (Detergent)
3      0.40      (Eggs)
4      0.20      (Bread)
5      0.45      (Shampoo)
6      0.25      (Coffee)
7      0.25      (Soap)
8      0.30      (Cereal)
9      0.25      (Toothpaste)
10     0.20      (Milk, Diapers)
11     0.20      (Eggs, Milk)
12     0.20      (Eggs, Shampoo)
13     0.25      (Detergent, Shampoo)
14     0.20      (Eggs, Soap)
FP-Growth Rules:
antecedents  consequents  antecedent support  consequent support  support  \
0      (Milk)      (Diapers)      0.40      0.30      0.20
1      (Diapers)      (Milk)      0.30      0.40      0.20
2      (Eggs)      (Milk)      0.40      0.40      0.20
3      (Milk)      (Eggs)      0.40      0.40      0.20
4      (Eggs)      (Shampoo)      0.40      0.45      0.20
5      (Detergent)      (Shampoo)      0.50      0.45      0.25
6      (Shampoo)      (Detergent)      0.45      0.50      0.25
7      (Eggs)      (Soap)      0.40      0.25      0.20
8      (Soap)      (Eggs)      0.25      0.40      0.20

confidence  lift  leverage  conviction  zhangs_metric
0      0.500000  1.666667  0.080      1.400      0.666667
1      0.666667  1.666667  0.080      1.800      0.571429
2      0.500000  1.250000  0.040      1.200      0.333333
3      0.500000  1.250000  0.040      1.200      0.333333
4      0.500000  1.111111  0.020      1.100      0.166667
5      0.500000  1.111111  0.025      1.100      0.200000
6      0.555556  1.111111  0.025      1.125      0.181818
7      0.500000  2.000000  0.100      1.500      0.833333
8      0.800000  2.000000  0.100      3.000      0.666667
FP-Growth Time: 0.0141s

Try another dataset? (yes/no): no
Exiting the program.

```

Fig 4: FP-Growth algorithm and rules

Other:

The source code (.py file) and data sets (.csv files) will be attached to the zip file.

Link to Git Repository:

https://github.com/aesha2492/DM_midtermproj